

Package ‘aocseq’

January 10, 2025

Type Package

Title aocseq 0.1.0: An R package for mixed cell type annotation following the activation of cells & sequencing.

Version 0.1.0

Author Mae Woods, Noah Crooks

Maintainer <mae.woods@bcm.edu>

Description aocseq is a suite of statistical tools for the analysis of multimodal immunosequencing data. The purpose of this statistical tool is to quantify single cell data and provide 1) A sample list of mixed cell types with accompanying data sheets to measure the number of different cell types that are present in the blood or tissue and 2) To detect, score and rank cells with optimal or desirable characteristics. Characteristics are defined by the gene expression of cells that are considered of interest because of their response to perturbation.

License BSD2

Encoding UTF-8

LazyData true

Imports Seurat, matrixStats, readr, Rcpp, stats, BiocParallel, Matrix, MASS, VGAM, bbmle, gamlss, maxLik, pscl, rjson

RoxygenNote 7.3.2

R topics documented:

AnnotateCellTypes	2
CellTypeLoop	3
ClassifyCells	4
ClassifyCellTypes	5
CombineData	6
ConstructTree	7
DEsingle	8
EStep	10
GetGeneSignature	10
GetSpecificCells	11
GetTCR	12
GMMDemux	12
IsoForest	13
MakeReference	14
MStep	15

NormalizationScore 15

PercentOutlier 16

QCPlot 16

SaveHeatmap 17

SetCellType 18

UMAPReduce 18

Index **20**

AnnotateCellTypes	<i>List cell types and the proportion of cells expressing a choice of marker genes at normalized counts greater than the control</i>
-------------------	--

Description

This function imports a Seurat object list processed by the aocseq::CombineData function and produces a spreadsheet of cell types ranked by frequency. The table displays the percentage of CD4 and CD8 cells expressing a choice of marker genes at normalized counts greater than the control.

Usage

```
AnnotateCellTypes(  
  cell.data,  
  goi,  
  cell.type = "cdr3_na",  
  goi.threshold = 0.975,  
  TCR = TRUE,  
  index.control.input = 1,  
  conditions = 1,  
  path = "",  
  names.spreadsheet = c(-1),  
  preset = 1,  
  threshold.entry = FALSE  
)
```

Arguments

cell.data	A Seurat object pre-processed with aocseq::CombineData.
goi	A marker of interest.
cell.type	Clone identifier, e.g. cdr3_na, phenotype.
goi.threshold	Quantile that determines high counts per cell.
TCR	True or false if true, adds amino acid sequence of TCR if cell.type is set to cdr3_na.
index.control.input	Index of the control sequencing sample for each condition.
conditions	Number of experiments within ClonalData object
path	String. Directory of the table output.
names.spreadsheet	List. Control and sample names.

preset	Bool. If set to 1, the control is part of the dataset. If 0, a threshold value for each condition must be set.
threshold.entry	List. List of thresholds for each condition if preset is 0.

Value

A data frame containing clonotypes and summary statistics of transcript expression.

CellTypeLoop	<i>Return a data frame of meta data associated with single cell data sets that contains a distance from a reference that is determined using the isolation forest algorithm</i>
--------------	---

Description

This function will return the percentage of cells that are outliers when compared to a reference data set.

Usage

```
CellTypeLoop(
  cell.data,
  cell.types,
  reference.data,
  genes,
  ntrees,
  maxheight,
  solver = TRUE,
  subsample.count = ncol(reference.data) + 1
)
```

Arguments

cell.data	A Seurat object containing cells that are to be assigned a distance.
cell.types	A list of the cell types (currently this is clonotypes).
reference.data	Normalized reference data.
genes	Genes to pull from the query data set.
ntrees	Number of trees used for the isolation forest.
maxheight	Maximum height used for the isolation forest.
solver	True or false If true an Rcpp implementation of the software is used
subsample.count	Sub sampling number for isolation forest.

Value

A data frame containing the mean normalized isolation forest score for each cell type.

ClassifyCells	<i>Cell classifier function that assigns a mean distance per cell type based on several metrics used for single cell RNA sequencing classification.</i>
---------------	---

Description

This function will compute a classification for each single cell in samples that have been activated and sequenced. Takes as input two Seurat objects, one containing the query cells and another containing the reference. A gene list used to construct the distance and output path for plotting and storing the mean distance per cell type. Other parameters are listed for debugging, but can be left as default values.

Usage

```
ClassifyCells(
  output.array,
  reference.data,
  gene.list,
  cell.types = c(),
  distance = 0,
  scramble = FALSE,
  withSCT = FALSE,
  ntrees = 10,
  maxheight = 20,
  PRINT = FALSE,
  file.path = ".",
  verbose = TRUE
)
```

Arguments

<code>output.array</code>	A Seurat object containing cells that are to be assigned an Rscore.
<code>reference.data</code>	Matrix of doubles. Reference data used to calculate the Rscore.
<code>gene.list</code>	Vector. List of genes in the gene signature.
<code>cell.types</code>	Vector. List of cell types for classification.
<code>distance</code>	Can be set to 0,1,2 or 3. Choice of 0 returns the Mahalanobis distance, 1 returns the taxicab distance, 2 returns the z-score and 3 returns the isolation forest distance.
<code>scramble</code>	Bool. To asses the effect of the gene signature on the Rscore, select a random set of genes and compute their distance from the reference data.
<code>withSCT</code>	Print progress bars and output
<code>ntrees</code>	Print progress bars and output
<code>maxheight</code>	Print progress bars and output
<code>PRINT</code>	True or false
<code>file.path</code>	path to save metadata as spreadsheet if PRINT is set to TRUE
<code>verbose</code>	Print progress bars and output

Value

A Seurat object containing aocseq cell classification metadata.

ClassifyCellTypes	<i>Combine single cell data and annotate with cell labels based on functionality</i>
-------------------	--

Description

This function will read in Seurat objects that have been processed with aocseq, and accompanying annotation tables to classify annotation.tab using distance scores. Takes as input gene expression from scRNAseq and VDJ enrichment. Outputs a classification table that lists samples and each clonotype or cell type and the classification.

Usage

```
ClassifyCellTypes(
  cell.data,
  annotation.tab,
  cell.type = "cdr3_na",
  within.sample = FALSE,
  method = "ZINB",
  goi = c("IFNG"),
  percentile = 0.01,
  path = "",
  path.glist = "",
  reference = matrix(nrow = 1, ncol = 1),
  verbose = TRUE
)
```

Arguments

cell.data	A seurat object. Pre-processed with aocseq.
annotation.tab	Table. An annotation table from the aocseq package.
cell.type	Cell types to classify each sub population.
within.sample	True or False. Set to TRUE for comparison between control and test data. For within sample cell type annotation, set to FALSE.
method	Choice of "ZINB", "Mahalanobis", "z-score", "IsoForest" and "Taxi cab". "ZINB" can be used without a reference data set.
goi	Gene. Gene of interest (goi). For use with "ZINB" classification.
percentile	Double. Percentile cutoff for distance based scoring metrics.
path	Directory for output tables.
verbose	Print progress bars and output.

Value

A Seurat object list containing metadata and VDJ annotations.

CombineData	<i>Combine single cell data and annotate with cell labels based on functionality</i>
-------------	--

Description

This function will read in 10X genomics outputs from cellranger and combine multiple Seurat objects, assigning labels based on additional tables stored in csv format. Takes as input gene expression from scRNAseq and VDJ enrichment. A gene list used to estimate specificity can be chosen and is application specific. Other parameters are listed for debugging, but can be left as default values.

Usage

```
CombineData(
  gex.path,
  marker.gene,
  vdj.path = c(),
  threshold.cutoff = 0.975,
  file.saved = "samples.rds",
  index.control = c(-1),
  sample.name = c(-1),
  preset = 1,
  threshold.entry = 0,
  demultiplex = FALSE,
  demultiplex.index = c(),
  nameshashtags = c(),
  n.ht.per.sample = 1,
  tenX_conversion = "true",
  nFeature_RNA_lower = 100,
  nFeature_RNA_upper = 10000,
  nvariable_features = 3000,
  percent.mt_upper = 5,
  data.input = "raw",
  upperQ = 0.95,
  lowerQ = 0.05,
  verbose = TRUE,
  QC_plots = FALSE
)
```

Arguments

<code>gex.path</code>	path to gene expression data in the format of cellranger feature barcode matrices.
<code>marker.gene</code>	list of marker genes used for specificity analysis.
<code>vdj.path</code>	path to VDJ expression data in the format of cellranger csv files.
<code>threshold.cutoff</code>	list of marker genes used for specificity analysis.
<code>file.saved</code>	directory for storing Seurat object RDS files.
<code>index.control</code>	Index of the control sequencing sample.

<code>sample.name</code>	Names that will be added to each sample <code>orig.ident</code> .
<code>preset</code>	If set to 1, <code>preset</code> determines the threshold for cutoff if a control is included in the assay. If set to 0 a cutoff is set by the <code>threshold.entry</code> , which defines what value is high for all samples. If set to 2 a cutoff is set per sample.
<code>threshold.entry</code>	Double. Sets the percentile above which gene expression is labelled as high.
<code>demultiplex</code>	Boolean value used to indicate if the data was hashtagged and therefore requires to be split.
<code>demultiplex.index</code>	Indexes to select columns with hashtag counts in the 10X genomics Seurat object assay. This depends on the 10X genomics chemistry and output from <code>cellranger</code> .
<code>n.ht.per.sample</code>	Number of hashtags used per sample (only set if hashtagging has been used).
<code>tenX_conversion</code>	Parameter for 10X compatability.
<code>nFeature_RNA_lower</code>	Threshold for the minimum number of unique gene identifiers detected in a single cell.
<code>nFeature_RNA_upper</code>	Threshold for the maximum number of unique gene identifiers detected in a single cell.
<code>nvariable_features</code>	Threshold for the maximum total number of unique gene identifiers detected in a single cell for dimension reduction.
<code>percent.mt_upper</code>	Threshold for the maximum percentage of unique mitochondrial gene identifiers detected in a single cell.
<code>verbose</code>	Print progress bars and output
<code>QC_plots</code>	Print progress bars and output
<code>names.hashtag</code>	hashtag sample names.

Value

A Seurat object list containing metadata and VDJ annotations.

ConstructTree

Build a binary tree from a numerical data frame

Description

This function will build a binary tree from a numerical data frame and return a data frame containing each unique data point from the input data and the height at which each data point becomes "isolated" (reaches an external node) or the `max_height`.

Usage

```
ConstructTree(data, current_height, max_height, kurtosis = TRUE)
```

Arguments

<code>data</code>	a numerical data frame
<code>current_height</code>	a parameter which tracks the current height of a given <code>construct_tree</code> instance, allows for recursive calling of the function
<code>max_height</code>	the maximum height the tree will grow to before stopping
<code>kurtosis</code>	splits the data based on the kurtosis over the maximum gene distribution across the cells in the input data.

Value

a data frame containing the data for each unique point in the input data and the height at which that point was isolated

DEsingle	<i>This is a copy of DEsingle that was lifted from the original package, see https://bioconductor.org/packages/release/bioc/html/DEsingle.html and included in the aocseq package with modifications for package compatibility DEsingle: Detecting differentially expressed genes from scRNA-seq data This function is used to detect differentially expressed genes between two specified groups of cells in a raw read counts matrix of single-cell RNA-seq (scRNA-seq) data. It takes a non-negative integer matrix of scRNA-seq raw read counts or a SingleCellExperiment object as input. So users should map the reads (obtained from sequencing libraries of the samples) to the corresponding genome and count the reads mapped to each gene according to the gene annotation to get the raw read counts matrix in advance.</i>
----------	---

Description

This is a copy of DEsingle that was lifted from the original package, see <https://bioconductor.org/packages/release/bioc/html/DEsingle.html> and included in the aocseq package with modifications for package compatibility DEsingle: Detecting differentially expressed genes from scRNA-seq data This function is used to detect differentially expressed genes between two specified groups of cells in a raw read counts matrix of single-cell RNA-seq (scRNA-seq) data. It takes a non-negative integer matrix of scRNA-seq raw read counts or a SingleCellExperiment object as input. So users should map the reads (obtained from sequencing libraries of the samples) to the corresponding genome and count the reads mapped to each gene according to the gene annotation to get the raw read counts matrix in advance.

Usage

```
DEsingle(counts, group, goi, parallel = FALSE, BPPARAM = bpparam())
```

Arguments

<code>counts</code>	A non-negative integer matrix of scRNA-seq raw read counts or a SingleCellExperiment object which contains the read counts matrix. The rows of the matrix are genes and columns are samples/cells.
<code>group</code>	A vector of factor which specifies the two groups to be compared, corresponding to the columns in the counts matrix.

parallel	If FALSE (default), no parallel computation is used; if TRUE, parallel computation using BiocParallel, with argument BPPARAM.
BPPARAM	An optional parameter object passed internally to bplapply when parallel=TRUE. If not specified, bpparam() (default) will be used.

Value

A data frame containing the differential expression (DE) analysis results, rows are genes and columns contain the following items:

- theta_1, theta_2, mu_1, mu_2, size_1, size_2, prob_1, prob_2: MLE of the zero-inflated negative binomial distribution's parameters of group 1 and group 2.
- total_mean_1, total_mean_2: Mean of read counts of group 1 and group 2.
- foldChange: total_mean_1/total_mean_2.
- norm_total_mean_1, norm_total_mean_2: Mean of normalized read counts of group 1 and group 2.
- norm_foldChange: norm_total_mean_1/norm_total_mean_2.
- chi2LR1: Chi-square statistic for hypothesis testing of H0.
- pvalue_LR2: P value of hypothesis testing of H20 (Used to determine the type of a DE gene).
- pvalue_LR3: P value of hypothesis testing of H30 (Used to determine the type of a DE gene).
- FDR_LR2: Adjusted P value of pvalue_LR2 using Benjamini & Hochberg's method (Used to determine the type of a DE gene).
- FDR_LR3: Adjusted P value of pvalue_LR3 using Benjamini & Hochberg's method (Used to determine the type of a DE gene).
- pvalue: P value of hypothesis testing of H0 (Used to determine whether a gene is a DE gene).
- pvalue.adj.FDR: Adjusted P value of H0's pvalue using Benjamini & Hochberg's method (Used to determine whether a gene is a DE gene).
- Remark: Record of abnormal program information.

Author(s)

Zhun Miao.

See Also

[DEtype](#), for the classification of differentially expressed genes found by [DEsingle](#).
[TestData](#), a test dataset for [DEsingle](#).

Examples

```
# Load test data for DEsingle
data(TestData)

# Specifying the two groups to be compared
# The sample number in group 1 and group 2 is 50 and 100 respectively
group <- factor(c(rep(1,50), rep(2,100)))

# Detecting the differentially expressed genes
results <- DEsingle(counts = counts, group = group)
```

```
# Dividing the differentially expressed genes into 3 categories
results.classified <- DType(results = results, threshold = 0.05)
```

EStep	<i>EStep</i>
-------	--------------

Description

EStep

Usage

```
EStep(x, mu.vector, sd.vector, alpha.vector)
```

Arguments

sd.vector	Vector containing the mean of each component
alpha.vector	Vector containing the mixing weights of each component

Value

Named list containing the loglik and posterior.df

GetGeneSignature	<i>Export differential gene expression from several different samples and phenotypes</i>
------------------	--

Description

This function will take a set of annotation spreadsheets and export differential expression between different cell types labelled by their marker genes. Currently the phenotypic separation is CD4 and CD8, but the reader is encouraged to generate as many subsets as needed for downstream analysis.

Usage

```
GetGeneSignature(
  cell.data,
  clonotype.path,
  save.dir,
  goi,
  FClim = 0,
  specific.pval = 10-5,
  bystander.pval = 10-2,
  set.min.pct = 0.25,
  logfc.threshold = 0
)
```

Arguments

<code>cell.data</code>	A Seurat object pre-processed with <code>aocseq::CombineData</code> .
<code>clonotype.path</code>	Character array. Directory of an <code>aocseq</code> clonotype annotation table.
<code>save.dir</code>	Directory for storing differentially expressed genes.
<code>goi</code>	Character array. Gene name, a marker of interest.
<code>verbose</code>	Print progress

Value

return an assay containing predicted expression value in the data slot

<code>GetSpecificCells</code>	<i>Subsets a SEURAT object based on a list of meta data for the CD4 and CD8 subsets</i>
-------------------------------	---

Description

This function will read in Seurat objects processed by `traceseq::AnnotateClonotypes` or any other software based on cell annotation and subset the SEURAT array accordingly

Usage

```
GetSpecificCells(cell.data, expression, phenotype, tcrseq, goi_v)
```

Arguments

<code>cell.data</code>	A Seurat object.
<code>expression</code>	Choice of "high" or "unassigned".
<code>phenotype</code>	CD4 or CD8 clasification.
<code>tcrseq</code>	List of TCRs to mark clonotypes included in the analysis.
<code>goi_v</code>	Gene of interest.

Value

A subset of a Seurat object processed with `aocseq`.

GetTCR	<i>Code to pull the full length TCR sequence from 10X genomics all_contig_annotations.json file</i>
--------	---

Description

This function will import a set of single cell barcodes, extract the full length TCR transcript details and output the details ranked by the cell with the greatest functional score.

Usage

```
GetTCR(
  barcodes,
  contig.annoations,
  json.path = ".",
  save.dir = ".",
  score = "",
  verbose = TRUE
)
```

Arguments

barcodes	Barcodes used to identify cells in full contig annotations JSON file.
contig.annoations	contig_annotations.csv file.
json.path	Directory where JSON file is saved.
save.dir	Directory where full length TCR transcripts will be saved.
score	Orders the cells be classification score.
verbose	Print progress bars and output

Value

Void

GMMDemux	<i>Flexible implementation of the GMM demux algorithm</i>
----------	---

Description

Demultiplex arbitrary number of hashtags

Usage

```
GMMDemux(
  s.name,
  data,
  hashtag.index,
  nameshashtags,
  set.col.name = "Hashtags",
  Seurat_Object = FALSE
)
```

Arguments

s.name	Input data. 10x object with hashtags
data	Input data. 10x object with hashtags
hashtag.index	list containing positions of hashtags to be demultiplexed
nameshashtags	Subsets of the sample for new orig.ident
set.col.name	Subsets of the sample for new orig.ident
Seurat_Object	Name of the sample

Value

A Seurat object that has been split by hastagged demultiplexing.

IsoForest	<i>Build many trees on samples from data and average the heights</i>
-----------	--

Description

Build many trees on samples from data and average the heights

Usage

```
IsoForest(
  data,
  num_trees,
  max_height,
  subsample_count = nrow(data),
  kurtosis_param = TRUE
)
```

Arguments

data	a numerical data frame
num_trees	the number of trees to build and average over
max_height	the maximum height each tree will grow to before stopping
subsample_count	the size of the random sample that a tree will be built on. Cannot be larger than the number of rows in the data
kurtosis_param	splits the data based on the kurtosis over the maximum gene distribution across the cells in the input data.

Value

a data frame containing the data for each unique point in the input data and the average height at which that point was isolated

MakeReference	<i>These are functions to produce reference datasets to compute a response score of activated or perturbed T cells.</i>
---------------	---

Description

This function returns a matrix of normalized gene expression along the rows and cells along the columns.

Usage

```
MakeReference(
  cell.data,
  gene.list,
  cell.type.list,
  celltype = cdr3_na,
  threshold = 0.975,
  n.inlist = 10,
  cellcycle = FALSE,
  SCT = FALSE,
  verbose = TRUE
)
```

Arguments

<code>cell.data</code>	List of Seurat objects. Seurat objects containing expression data processed. by aocseq that will be used to score query T cell response.
<code>gene.list</code>	List of genes to be included in the reference matrix.
<code>cell.type.list</code>	Character list. List of cell types to include in the reference data.
<code>celltype</code>	Cell type to access metadata of reference dataset.
<code>n.inlist</code>	Number of genes with high expression in each cell included in the reference matrix.
<code>cellcycle</code>	Adds Seurat cell cycle phase to reference data.
<code>SCT</code>	Returns reference matrix in sctransform space.
<code>verbose</code>	Print progress bars and output.

Value

A reference matrix.

MStep

Maximization Step of the EM Algorithm

Description

Update the Component Parameters

Usage

```
MStep(x, posterior.df)
```

Arguments

x	Input data.
posterior.df	Posterior probability data.frame.

Value

Named list containing the mean (mu), variance (var), and mixing weights (alpha) for each component.

NormalizationScore

This function normalizes the data from the R implementation of the isolation forest

Description

This function normalizes the data from the R implementation of the isolation forest

Usage

```
NormalizationScore(df)
```

Arguments

df	A data frame containing isolation forest distances
----	--

Value

Normalized values

PercentOutlier	<i>Return a data frame of meta data associated with single cell data sets that contains a distance from a reference</i>
----------------	---

Description

This function will return the percentage of cells that are outliers when compared to a reference data set.

Usage

```
PercentOutlier(
  cell.data,
  reference.data,
  genes,
  ntrees,
  maxheight,
  subsample.count = ncol(reference.data) + 1
)
```

Arguments

cell.data	A Seurat object containing cells that are to be assigned a distance.
reference.data	Normalized reference data.
genes	Genes to pull from the query dataset.
ntrees	Number of trees used for the isolation forest.
maxheight	Maximum height used for the isolation forest.
subsample.count	Subsampling number for isolation forest.

Value

A data frame with the percentage of cells that are outliers.

QCPlot	<i>QC plot</i>
--------	----------------

Description

This function will take a set of annotation spreadsheets and export differential expression between different cell types labelled by their marker genes. Currently the phenotypic separation is CD4 and CD8, but the reader is encouraged to generate as many subsets as needed for downstream analysis.

Usage

```
QCPlot(
  Clonal_Obs,
  nFeature_RNA_lower = 100,
  nFeature_RNA_upper = 10000,
  nCount_RNA_lower = 100,
  nCount_RNA_upper = 10000,
  percent.mt_upper = 5
)
```

Arguments

`Clonal_Obs` A Seurat object pre-processed with `aocseq::CombineData`.

`clonotype.path` Character array. Directory of an aocseq clonotype annotation table.

`save.dir` Directory for storing differentially expressed genes.

`goi` Character array. Gene name, a marker of interest.

`verbose` Print progress

Value

return an assay containing predicted expression value in the data slot

SaveHeatmap	<i>This function will take a set of annotation spreadsheets and export differential expression between different cell types labelled by their marker genes. Currently the phenotypic separation is CD4 and CD8, but the reader is encouraged to generate as many subsets as needed for downstream analysis.</i>
-------------	---

Description

This function will take a set of annotation spreadsheets and export differential expression between different cell types labelled by their marker genes. Currently the phenotypic separation is CD4 and CD8, but the reader is encouraged to generate as many subsets as needed for downstream analysis.

Usage

```
SaveHeatmap(
  Resting.cells,
  Act.ref,
  CellsInRef,
  path.glist,
  heatmap.path,
  verbose = TRUE
)
```

Arguments

<code>verbose</code>	Print progress
<code>Clonal_Obs</code>	A Seurat object pre-processed with <code>aocseq::CombineData</code> .
<code>clonotype.path</code>	Character array. Directory of an aocseq clonotype annotation table.
<code>save.dir</code>	Directory for storing differentially expressed genes.
<code>goi</code>	Character array. Gene name, a marker of interest.

Value

return an assay containing predicted expression value in the data slot

<code>SetCellType</code>	<i>This function sets the <code>cell.types</code> as meta data.</i>
--------------------------	---

Description

This function sets the `cell.types` as meta data.

Usage

```
SetCellType(cell.data, cell.types)
```

Arguments

<code>cell.data</code>	A Seurat Object.
<code>cell.types</code>	Single cell metadata.

Value

A Seurat Object.

<code>UMAPReduce</code>	<i>Export differential gene expression from several different samples and phenotypes</i>
-------------------------	--

Description

This function will take a set of annotation spreadsheets and export differential expression between different cell types labelled by their marker genes. Currently the phenotypic separation is CD4 and CD8, but the reader is encouraged to generate as many subsets as needed for downstream analysis.

Usage

```
UMAPReduce(  
  Clonal_Obs,  
  save.dir = ".",  
  clonotypes = c(),  
  ident.list = c("orig.ident"),  
  rseed = 356,  
  ur.nfeatures = 500,  
  preload = FALSE,  
  preloadpath = "."  
)
```

Arguments

Clonal_Obs	A Seurat object pre-processed with aocseq::CombineData.
save.dir	Directory for storing differentially expressed genes.
clonotype.path	Character array. Directory of an aocseq clonotype annotation table.
goi	Character array. Gene name, a marker of interest.
verbose	Print progress

Value

return an assay containing predicted expression value in the data slot

Index

- * **Annotation & quality control**

- CombineData, [6](#)

- * **Annotation**

- AnnotateCellTypes, [2](#)

- SetCellType, [18](#)

- * **Genomics**

- GetTCR, [12](#)

- * **Routine functions**

- ConstructTree, [7](#)

- EStep, [10](#)

- MStep, [15](#)

- NormalizationScore, [15](#)

- * **Routine functions**

- IsoForest, [13](#)

- * **Single cell analysis**

- CellTypeLoop, [3](#)

- ClassifyCells, [4](#)

- ClassifyCellTypes, [5](#)

- GetGeneSignature, [10](#)

- GetSpecificCells, [11](#)

- MakeReference, [14](#)

- PercentOutlier, [16](#)

- * **Statistical inference**

- DEsingle, [8](#)

- GMMDemux, [12](#)

- * **Visualization**

- QCPlot, [16](#)

- SaveHeatmap, [17](#)

- UMAPReduce, [18](#)

AnnotateCellTypes, [2](#)

bplapply, [9](#)

bpparam, [9](#)

CellTypeLoop, [3](#)

ClassifyCells, [4](#)

ClassifyCellTypes, [5](#)

CombineData, [6](#)

ConstructTree, [7](#)

DEsingle, [8](#), [9](#)

DEtype, [9](#)

EStep, [10](#)

GetGeneSignature, [10](#)

GetSpecificCells, [11](#)

GetTCR, [12](#)

GMMDemux, [12](#)

IsoForest, [13](#)

MakeReference, [14](#)

MStep, [15](#)

NormalizationScore, [15](#)

PercentOutlier, [16](#)

QCPlot, [16](#)

SaveHeatmap, [17](#)

SetCellType, [18](#)

TestData, [9](#)

UMAPReduce, [18](#)